

Advanced Variants

Turunan penting dari pattern inti: monotonic stack, binary search on answer, dan keluarga DP.

Tujuan dokumen ini: bukan menghafal tanda-tanda soal, tapi memahami gerak berpikir pattern. Setiap pattern dijelaskan dengan contoh dunia nyata, contoh teknologi, konsep komputasi, alasan kenapa contoh itu sama dengan pattern, dan template Dart minimal.

Cara Membaca Dokumen Ini

Jangan baca seperti kamus keyword. Baca seperti latihan membangun intuisi. Pattern LeetCode itu mirip alat kerja: tiap alat punya cara bergerak, situasi natural, dan bentuk data yang cocok.

Untuk tiap pattern, pakai urutan baca ini: makna manusiawi -> contoh dunia nyata -> contoh teknologi -> konsep komputasi -> bentuk LeetCode -> template Dart -> pertanyaan cek pemahaman.

Pattern	Makna inti
Monotonic Stack	stack yang menjaga urutan naik/turun
Binary Search on Answer	mencari jawaban minimum/maksimum dengan can(x)
Knapsack DP	ambil/tidak ambil item untuk target
LIS DP	membangun urutan meningkat terbaik
2D DP	state dua dimensi
Tree DP	DP yang berjalan di tree
State Machine DP	DP dengan mode/status

Monotonic Stack - stack yang menjaga urutan naik/turun

Makna manusiawinya

Monotonic stack adalah stack yang isinya dijaga tetap naik atau turun. Ia dipakai untuk mencari elemen berikutnya yang lebih besar/kecil secara efisien.

Contoh nyata non-teknologi

Menunggu orang lebih tinggi di antrean

Lu punya daftar orang. Untuk tiap orang, ingin tahu kapan setelahnya ada orang lebih tinggi. Orang yang belum menemukan orang lebih tinggi disimpan.

Harga saham menunggu breakout

Setiap hari yang belum menemukan harga lebih tinggi disimpan. Saat harga baru lebih tinggi, beberapa hari sebelumnya langsung terjawab.

Contoh nyata di dunia teknologi

Daily Temperatures

Index hari disimpan sampai ada suhu lebih panas.

Largest Rectangle Histogram

Stack menjaga bar dengan tinggi meningkat untuk menghitung area saat turun.

Konsep komputasinya

Secara komputasi, item masuk sekali dan keluar sekali, jadi $O(n)$. Stack menyimpan kandidat yang belum selesai, sambil menjaga urutan monotonic.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena kita menunggu event masa depan yang menyelesaikan banyak item lama sekaligus.

Bentuknya di LeetCode

- Daily Temperatures
- Next Greater Element
- Largest Rectangle in Histogram
- Remove K Digits

Template Dart minimal

```
List<int> dailyTemperatures(List<int> temperatures) {  
  final answer = List.filled(temperatures.length, 0);  
  final stack = <int>[]; // index dengan suhu yang belum menemukan suhu lebih panas  
  
  for (int i = 0; i < temperatures.length; i++) {  
    while (stack.isNotEmpty && temperatures[i] > temperatures[stack.last]) {  
      final prev = stack.removeLast();  
      answer[prev] = i - prev;  
    }  
    stack.add(i);  
  }  
  return answer;  
}
```

Kalau masih bingung, tanya begini

- Apakah gue mencari next greater/smaller?
- Apakah item lama bisa diselesaikan oleh item baru?
- Stack harus increasing atau decreasing?

Mini drill pemahaman

1. Kenapa setiap index hanya pop sekali?
2. Apa isi stack: value atau index?
3. Kapan pakai monotonic queue bukan stack?

Binary Search on Answer - mencari jawaban minimum/maksimum dengan can(x)

Makna manusiawinya

Ini binary search yang ruang pencariannya bukan array, tapi kemungkinan jawaban. Selama bisa membuat fungsi can(x) yang monotonic, kita bisa mencari batas.

Contoh nyata non-teknologi

Mencari budget minimum

Budget X cukup untuk event. Kalau X cukup, budget lebih besar pasti cukup. Cari budget minimum yang cukup.

Mencari waktu belajar minimal

Kalau 3 jam per hari cukup untuk target, 4 jam juga cukup. Cari minimum jam yang masih cukup.

Contoh nyata di dunia teknologi

Capacity To Ship Packages

Jika kapasitas kapal X cukup, kapasitas lebih besar pasti cukup.

Koko Eating Bananas

Jika speed X cukup, speed lebih tinggi pasti cukup.

Konsep komputasinya

Secara komputasi, kunci utamanya can(x). Binary search hanya mesin; can(x) adalah model masalah. Kalau can salah, seluruh solusi salah.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena ada threshold. Kita mencari titik pertama yang memenuhi constraint.

Bentuknya di LeetCode

- Koko Eating Bananas
- Ship Within D Days
- Minimized Maximum
- Split Array Largest Sum

Template Dart minimal

```
int minEatingSpeed(List<int> piles, int h) {
  int low = 1;
  int high = piles.reduce((a, b) => a > b ? a : b);

  bool can(int speed) {
    int hours = 0;
    for (final pile in piles) {
      hours += (pile + speed - 1) ~/ speed;
    }
    return hours <= h;
  }

  while (low < high) {
```

```
        final mid = (low + high) ~/ 2;
        if (can(mid)) high = mid;
        else low = mid + 1;
    }
    return low;
}
```

Kalau masih bingung, tanya begini

- Apa answer space-nya?
- Low/high realistis apa?
- Apa can(x)?
- Apakah monotonic?

Mini drill pemahaman

1. Buat can(capacity) untuk shipping.
2. Kenapa low/high penting?
3. Apa beda return first true dan last true?

Knapsack DP - ambil/tidak ambil item untuk target

Makna manusiawinya

Knapsack adalah DP pilihan item. Untuk tiap item, lu memutuskan ambil atau tidak, dengan constraint kapasitas/target.

Contoh nyata non-teknologi

Packing koper

Setiap barang punya berat dan nilai. Kapasitas koper terbatas. Ambil/tidak ambil barang untuk hasil terbaik.

Menyusun budget fitur

Setiap fitur punya biaya dan impact. Budget terbatas. Pilih subset fitur.

Contoh nyata di dunia teknologi

Partition Equal Subset Sum

Cari subset dengan jumlah target.

Coin Change variants

Pilih koin untuk mencapai jumlah tertentu.

Konsep komputasinya

Secara komputasi, state sering $dp[sum]$ atau $dp[i][sum]$. Iterasi mundur dipakai untuk 0/1 knapsack agar item tidak dipakai berkali-kali.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena keputusan setiap item bercabang: ambil atau skip. Target membatasi kombinasi valid.

Bentuknya di LeetCode

- Partition Equal Subset Sum
- Target Sum
- Last Stone Weight II
- Coin Change

Template Dart minimal

```
bool canPartition(List<int> nums) {  
  final total = nums.fold(0, (a, b) => a + b);  
  if (total.isOdd) return false;  
  final target = total ~/ 2;  
  final dp = List.filled(target + 1, false);  
  dp[0] = true;  
  
  for (final num in nums) {  
    for (int s = target; s >= num; s--) {  
      dp[s] = dp[s] || dp[s - num];  
    }  
  }  
  return dp[target];  
}
```

Kalau masih bingung, tanya begini

- Apakah ada item dan target/capacity?
- Item boleh dipakai sekali atau berkali-kali?
- Apakah menghitung bisa/tidak, minimum, maximum, atau jumlah cara?

Mini drill pemahaman

1. Kenapa loop sum mundur untuk 0/1?
2. Apa beda coin change minimum dan count ways?
3. Modelkan pilih fitur sebagai knapsack.

LIS DP - membangun urutan meningkat terbaik

Makna manusiawinya

LIS mencari subsequence meningkat terpanjang. Intuisinya: simpan ekor terkecil untuk panjang tertentu agar peluang masa depan lebih besar.

Contoh nyata non-teknologi

Membangun karier bertahap

Untuk mencapai level tinggi, jalur dengan ending lebih rendah tapi panjang sama lebih fleksibel untuk naik lagi.

Menyusun kartu angka naik

Kalau punya dua urutan sama panjang, yang berakhir dengan angka lebih kecil lebih bagus karena lebih mudah ditambah angka berikutnya.

Contoh nyata di dunia teknologi

Russian Doll Envelopes

Setelah sorting, cari chain ukuran meningkat.

Version progression

Mencari progres peningkatan dari data historis.

Konsep komputasinya

Ada versi $O(n^2)$ DP dan $O(n \log n)$ dengan binary search tails. tails bukan subsequence final, tapi representasi peluang terbaik.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena kita ingin urutan bertumbuh, dan ending kecil memberi ruang masa depan.

Bentuknya di LeetCode

- Longest Increasing Subsequence
- Number of LIS
- Russian Doll Envelopes

Template Dart minimal

```
int lengthOfLIS(List<int> nums) {  
  final tails = <int>[];  
  
  for (final x in nums) {  
    int l = 0, r = tails.length;  
    while (l < r) {  
      final m = (l + r) ~/ 2;  
      if (tails[m] < x) l = m + 1;  
      else r = m;  
    }  
    if (l == tails.length) tails.add(x);  
    else tails[l] = x;  
  }  
  return tails.length;  
}
```

Kalau masih bingung, tanya begini

- Apakah urutan asli harus dipertahankan?
- Apakah contiguous atau subsequence?
- Apakah tails bisa dipakai?

Mini drill pemahaman

1. Kenapa ending kecil lebih baik?
2. Apa beda subarray dan subsequence?
3. Kenapa sorting penting di Russian Doll?

2D DP - state dua dimensi

Makna manusiawinya

2D DP dipakai ketika subproblem membutuhkan dua koordinat/index/dimensi. Biasanya dua string, grid, atau dua batas.

Contoh nyata non-teknologi

Membandingkan dua dokumen

Kita melihat progress di dokumen A dan dokumen B sekaligus. State butuh posisi di dua dokumen.

Berjalan di grid

Jumlah cara ke cell tergantung baris dan kolom.

Contoh nyata di dunia teknologi

Longest Common Subsequence

State $dp[i][j]$ untuk prefix $text1$ dan $text2$.

Unique Paths

State $dp[r][c]$ untuk cara sampai cell tertentu.

Konsep komputasinya

Secara komputasi, $dp[i][j]$ harus punya makna jelas. Transition biasanya dari kiri, atas, diagonal, atau kombinasi state sekitar.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena satu angka index tidak cukup. Kita perlu mengetahui posisi di dua sumbu.

Bentuknya di LeetCode

- Unique Paths
- LCS
- Edit Distance
- Interleaving String
- Grid DP

Template Dart minimal

```
int uniquePaths(int m, int n) {  
  final dp = List.generate(m, (_) => List.filled(n, 1));  
  for (int r = 1; r < m; r++) {  
    for (int c = 1; c < n; c++) {  
      dp[r][c] = dp[r - 1][c] + dp[r][c - 1];  
    }  
  }  
  return dp[m - 1][n - 1];  
}
```

Kalau masih bingung, tanya begini

- Apa arti $dp[i][j]$?

- Dari state mana ia dibangun?
- Apa base row/column?
- Apakah bisa dioptimasi ke 1D?

Mini drill pemahaman

1. Definisikan $dp[i][j]$ untuk LCS.
2. Kenapa unique paths dari atas+kiri?
3. Apa risiko definisi state yang kabur?

Tree DP - DP yang berjalan di tree

Makna manusiawinya

Tree DP menyimpan beberapa jawaban per node, biasanya karena keputusan di node memengaruhi anak atau parent.

Contoh nyata non-teknologi

Memilih tim tanpa atasan langsung

Kalau memilih manager, mungkin tidak boleh memilih bawahan langsung. Tiap node perlu state dipilih/tidak dipilih.

Menghitung nilai organisasi

Nilai suatu divisi tergantung nilai sub-divisi.

Contoh nyata di dunia teknologi

House Robber III

Untuk tiap node, simpan hasil jika node diambil dan jika node dilewati.

Binary Tree Maximum Path Sum

Jawaban global dan kontribusi ke parent berbeda.

Konsep komputasinya

Secara komputasi, DFS postorder sangat natural. Anak dihitung dulu, lalu parent menggunakan hasil anak.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena keputusan parent dan child saling bergantung. Kita perlu lebih dari satu state per node.

Bentuknya di LeetCode

- House Robber III
- Binary Tree Maximum Path Sum
- Diameter of a Binary Tree
- Longest Univalue Path

Template Dart minimal

```
List<int> robTree(TreeNode? root) {  
  if (root == null) return [0, 0]; // [skip, take]  
  final left = robTree(root.left);  
  final right = robTree(root.right);  
  
  final take = root.val + left[0] + right[0];  
  final skip = (left[0] > left[1] ? left[0] : left[1]) +  
               (right[0] > right[1] ? right[0] : right[1]);  
  return [skip, take];  
}
```

Kalau masih bingung, tanya begini

- Apa state per node?

- Apakah parent butuh jawaban anak?
- Apakah jawaban global berbeda dari return value?

Mini drill pemahaman

1. Untuk House Robber III, apa arti skip/take?
2. Kenapa postorder natural?
3. Kapan perlu variable global?

State Machine DP - DP dengan mode/status

Makna manusiawinya

State machine DP dipakai ketika posisi saja tidak cukup; kita juga perlu tahu mode saat ini: sedang pegang saham, cooldown, bebas membeli, dan sebagainya.

Contoh nyata non-teknologi

Mengelola energi harian

Status lu bisa kerja, istirahat, atau recovery. Keputusan hari ini tergantung status kemarin.

Proses order

Order bisa pending, paid, shipped, completed. Transisi status menentukan aksi valid.

Contoh nyata di dunia teknologi

Stock buy/sell

State holding dan cash. Keuntungan tergantung apakah sedang memegang saham.

Cooldown problems

Setelah menjual, ada state cooldown sebelum boleh beli lagi.

Konsep komputasinya

Secara komputasi, state machine DP mendefinisikan mode dan transisi antar mode. Jawaban terbaik diperbarui per langkah waktu/index.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena aturan aksi bergantung pada status. Tanpa status, model masalah hilang.

Bentuknya di LeetCode

- Best Time to Buy/Sell Stock variants
- Stock with cooldown
- Stock with fee
- Paint House variants

Template Dart minimal

```
int maxProfit(List<int> prices) {  
  int hold = -prices[0];  
  int cash = 0;  
  
  for (int i = 1; i < prices.length; i++) {  
    final newCash = cash > hold + prices[i] ? cash : hold + prices[i];  
    final newHold = hold > cash - prices[i] ? hold : cash - prices[i];  
    cash = newCash;  
    hold = newHold;  
  }  
  return cash;  
}
```

Kalau masih bingung, tanya begini

- Mode/status apa saja?

- Transisi valid antar status apa?
- Apa nilai yang disimpan per status?
- Jawaban akhir status mana?

Mini drill pemahaman

1. Untuk stock, kenapa perlu hold dan cash?
2. Apa transisi buy/sell?
3. Kapan cooldown menjadi state tambahan?

Penutup

Mastery bukan berarti menghafal semua tanda soal. Mastery berarti saat melihat masalah, lu bisa memodelkan gerak masalahnya: apakah ia menyebar, menyelam, menjaga window, mencari boundary, mengingat key, menggabungkan kelompok, atau menyimpan subproblem.

Cara latihan: ambil satu pattern, pahami analoginya, kerjakan 3-5 soal mudah, tulis ulang konsepnya dengan bahasa sendiri, lalu campur dengan pattern lain supaya otak belajar mengenali bentuk masalah tanpa spoiler.